

CSE235 Week 14: Web APIs and FastAPI

The past few weeks we have looked at desktop GUIs using Tkinter. This week, we will introduce web programming in Python. Python has many web programming frameworks available. Django is one of the most popular, as it is a full stack framework (front and back end). However, modern web development has moved away from full stack programming on the server-side, and is instead focused on creating lightweight APIs for server-side programming. Python offers many frameworks for web API development, but we'll be focusing on FastAPI since it is easy to use and offers a convenient dashboard for testing out of the box.

Brief Overview of Web Programming

Web programming consists of two major components: front-end and back-end development. Front-end development focuses on creating the interfaces that users interact with using HTML, CSS, and JavaScript. Back-end programming focuses on storing and retrieving data from databases on a remote server. HTTP (hypertext transfer protocol) uses client-server model. The client, a web browser makes a request for a webpage when you click a link or enter a URL. The server responds to this request by returning the requested resource.

Web browsers can handle HTML (hypertext markup language), CSS (cascading style sheets), and JavaScript. HTML is used to describe the structure and content of the web page. CSS adds custom styling to the page. JavaScript is a programming language that is executed in the browser. These languages comprise the core of front-end web development.

Back-end web programming is the server-side programming that occurs. Web servers are responsible for storing and updating persistent data and managing logins. Historically, back-end programming has also carried the burden of rendering pages as well. Languages like PHP allowed the back-end PHP code to be mixed with HTML and JavaScript, which would generate the complete webpage that is returned the client's browser. In this case, the browser only has to render the

completed HTML document and apply CSS styles. Minimal JavaScript may have been used for front-end form verification and other small tasks. This style of web programming is referred to as thin client, thick server. The web server carries a large processing burden to handle both data processing and rendering, while the client has the minimal task of displaying the resulting page. More recently, this has flipped. As the number of users of the web has increased, web programming has transitioned to a thick client, thin server model. Modern front-end JavaScript frameworks like React, Angular, and Vue have moved web development to a point where the client now performs most of the rendering in the browser. The load on the server has been lightened, where servers are now primarily responsible for storing and returning data.

An API is an abstract programming interface. In the context of web development, it refers to applications that perform basic database operations at the behest of a client, such as inserting data or returning the results of a data request. CRUD refers to create, retrieve, update, and delete: the four main tasks most web APIs are responsible for. These correspond to the SQL `INSERT INTO`, `SELECT`, `UPDATE`, and `DELETE FROM` queries, respectively. REST stands for Representational State Transfer. REST APIs define ways for applications to interact with each other over HTTP by mapping the HTTP request methods to CRUD operations:

- GET: an HTTP GET request corresponds to the R in CRUD. It is a request to retrieve data
- POST: an HTTP POST request corresponds to the C in CRUD. It is a request to create data
- PATCH and PUT: technically the HTTP PATCH request is a partial update to an existing data item, and PUT is a full replacement of an existing item. They correspond to the U in CRUD
- DELETE: an HTTP DELETE request corresponds to the D in CRUD. It is a request to delete data

In many real world applications, the GET and POST methods see the most use and are overloaded to handle the other CRUD operations. Now, let's look at requests and responses.

Requests

A request is made by a client to the API. The request consists of the following components:

- Endpoint: The endpoint is the URL (eg. <https://www.google.com>) that is being sent a request
- Method: The HTTP request method being used
- Headers: Additional metadata like login tokens or cookies included with the request
- Body: Payload data (typically JSON) included with the

request. This is primarily used with POST and PUT requests.

Responses

The server responds to the request with the requested resource (if possible) and a response code. Response codes from 200-299 indicate success. Codes from 300-399 specify that a redirection took place.

400-499 indicate a request error. Codes 500-599 indicate a server error.

Some common response codes include:

- 200 OK: indicates the response is okay - 201 CREATED: indicates the resource was created (POST and some PUT) - 202 ACCEPTED: the request was accepted but not acted on - 301 MOVED PERMANENTLY: the resource being requested has moved, and the new location was returned - 400 BAD REQUEST: the server cannot or will not handle the request due to a error detected in the request - 401 UNAUTHORIZED: the client is not authenticated to access the requested resource - 403 FORBIDDEN: the client is not authorized to access the requested resource - 404 NOT FOUND: the requested resource cannot be located by the server - 405 METHOD NOT ALLOWED: the request method is recognized but not permitted for the requested resource - 500 INTERNAL SERVER ERROR: generic error message when a problem occurs on the server - 501 NOT IMPLEMENTED: the requested method is not implemented/handled by the server The resource returned can be anything from an HTTP file, an image, or a JSON data document. JSON is most commonly used in REST APIs.

URLs

URL stands for unified resource location. It is used to specify the endpoint of a resource being requested. A URL consists of the protocol, hostname, path, and query. URLs can also contain selectors after the query, but those are not relevant to API development. The protocol is typically HTTPS for APIs. HTTP may be used locally for testing, but is not used in production environments. The hostname is the name of the server hosting the resources being requested, such as

`www.google.com` or `www.amazon.com`. The path is the specific resource on a server being requested.

```
https://www.api.com/cool_resource/
^           ^           ^
|           |           |
protocol|           |
           hostname    |
                       path
```

After the path, a query string can be requested. The query string is typically used to add additional details to a get request. The query starts with a question mark and contains a list of key=value pairs separated by ampersands:

```
https://www.api.com/cool_resource?key1=value1&key2=value2
```

Request Bodies

The request body is (typically) JSON formatted data that is included with the request.

FastAPI

FastAPI is a Python package used for creating web APIs. As the name suggests, it is designed to allow you to do so quickly. Overall, it does achieve that goal. Of the many Python web frameworks I've used, FastAPI is the easiest and fastest. To install FastAPI, run:

```
pip install fastapi[standard]
```

To start using FastAPI, create a Python file called `main.py` and add the following code:

```
from fastapi import FastAPI

# create a FastAPI app object
app = FastAPI()

# the @app.get('/') means that this function
# handles get requests for the url '/'
@app.get('/')
def get_root():
    return "Hello World"
```

To run this, type `fastapi dev` into the terminal in the directory this file is located. This will start a local web server that you can use to access your code. By default, it starts the server on port 8000. Type the URL `localhost:8000` in your browser to visit the page. You should see a blank screen with "Hello World" displayed. Since FastAPI is designed for API development, it returns JSON by default. If we return a list or dictionary, it is automatically converted to JSON.

```
from fastapi import FastAPI

app = FastAPI()
```

```

@app.get('/')
def get_root():
    return "Hello World"

@app.get('/dict')
def get_dict():
    return { "key1": "value1", "key2": "value2" }

@app.get('/list')
def get_list():
    return [
        { "object1" : { "key1": "value1", "key2": "value2" } },
        { "object2" : { "key1": "value1", "key2": "value2" } },
        { "object3" : { "key1": "value1", "key2": "value2" } }
    ]

```

Adding parameters to the endpoint handler function allows you to read query parameters:

```

# this expects query parameters called key1 and key2
# and returns them as JSON
@app.get('/params')
def get_params(key1 = None, key2 = None):
    return { "key1" : key1, "key2" : key2 }

```

We can test this by going to `localhost:8000/params?key1=value1&key2=value2`. We can expect to see:

```

{"key1": "value1", "key2": "value2"}

```

The exact formatting will depend on how your browser is configured to show raw JSON.

We can also read URL parameters:

```

# put the url param in curly braces
# and add it as a function parameter
@app.get("url/{param}")
def get_url_param(param: str):
    return {"url_param": param}

```

FastAPI Testing Dashboard

FastAPI provides a dashboard for testing your API. To reach it, go to `localhost:8000/docs` in your browser. The dashboard provides a submenu for each API endpoint. From the submenu, you can see the

details of that endpoint and test it. The "Try it out" button allows you to enter parameters and/or body of the request.

Handling Post Requests

We've been using `@app.get()` to process GET requests. Similarly, `@app.post()` is used to specify endpoints that handle POST requests.

```
@app.post("/post-example")
def post_example():
    return { "key": "value" }
```

Unlike GET requests, which send their data through the URL query string, POST requests accept input via the request body, which is typically JSON. To read the request body, you need to specify the expected format of the JSON input. To do so, you can declare a class that represents the expected structure of the JSON data. The class must inherit from the `BaseModel` class defined in the `pydantic` package. Pydantic is a dependency of FastAPI, so it will already be installed with FastAPI. Classes that inherit from `BaseModel` specify their properties and their data types. Python allows data type annotations by placing a colon after a variable's name, followed by the expected data type. Pydantic will use this data type information to convert dictionaries/JSON data to/from the class. Below is an example class that inherits from `BaseModel`:

```
from pydantic import BaseModel

# example class that represents JSON data
# with an object at the top level
class ObjectExample(BaseModel):
    # specify the expected data types of each field
    # data1 is a string
    data1: str
    # data2 is an int
    data2: int
    # data 3 is an int with unknown contents
    # pydantic will interpret it as a list of strings
    data3: list

# example class that represents JSON data
# with an array at the top level
class ArrayExample(BaseModel):
    # here, we expect that the list contains ObjectExamples
    array: list[ObjectExample]
```

We can then specify the class as the type of the parameter for our post request.

```
# POST example that accepts JSON object
@app.post("/post-object")
def post_object(data: ObjectExample):
    print(f"data1: {data.data1}")
    print(f"data2: {data.data2}")
    print(f"data3: {data.data3}")
    return { "status" : "accpeted" }

# POST example that accepts JSON array
@app.post("/post-array")
def post_array(data: ArrayExample):
    for d in data.array:
        print(f"data1: {d.data1}")
        print(f"data2: {d.data2}")
        print(f"data3: {d.data3}")
    return { "status" : "accpeted" }
```